# Hockey Pool Analytics Pipeline

Architecture, Models, Diagnostics, and Results

Julian di Giovanni

2026-07-24

## Contents

# 1 Introduction

## 1.1 Project purpose

This project produces draft-day player rankings for a custom fantasy hockey pool held once per year. The pipeline is rebuilt annually from a shared codebase with minimal configuration changes, following the principle: *copy last season's config, update a few fields, run* `--stage all`.

The output is an Excel workbook and set of CSVs ranking every NHL player by predicted pool points for the upcoming season, broken down by position (forwards, defense, goalies) and combined into an overall ranking.

## 1.2 Pool scoring rules

Scoring rules are encoded in `<season>/config.yaml` under the `scoring:` key and are the same year-to-year. Table 1 summarises the current rules.

Table 1: Pool scoring rules (2026-27 season)

| Position | Event | Points |
|---|---|---|
| Forwards | Goal | 1 |
| | Assist | 1 |
| Defense | Goal | 1 |
| | Assist | 1 |
| | Plus/minus | 1 per unit |
| Goalies | Win | 1 |
| | Shutout win bonus (additive; 4 total) | 3 |
| | Best GAA among $\geq$40 GP qualifiers | +10 |
| | Best save% among $\geq$40 GP qualifiers | +10 |

## 1.3 Annual workflow

1. **July** (off-season): run `--stage all` to refresh historical data through the just-completed season and generate 2026-27 predictions.
2. **Draft day**: use the output rankings and XLSX workbook to make pick decisions.
3. **Following July**: copy `config.yaml` to the new season folder, update `season`, `season_id`, `roster_as_of_date`, and any `trade_overrides`. Run `--stage all`.

# 2 System Architecture

## 2.1 Folder layout

```
common/                shared library, reused every season
  config.py              SeasonConfig dataclass (loads config.yaml)
  scrape/
    nhl_api.py             NHL API client (stats, rosters, game logs)
    rosters.py             current-team lookup (handles off-season moves)
    sources/
      moneypuck.py           MoneyPuck CSV scraper (xG, shot quality,
          GSAx)
  clean/
    merge.py               merges skater/goalie/team data into model CSVs
```

```
  features/
    engineering.py        lag features, EB shrinkage, GSAx, leakage
       guard
  models/
    training.py           5-algorithm zoo, CV, feature selection, OOS
       eval
    forwards.py           forward target + feature config
    defense.py            defense target + feature config
    goalies.py            goalie target + feature config
    pool_ranking.py       scoring, blending, ranking, output writing
  diagnostics/
    reconcile.py          cross-source reconciliation (NHL API vs
       MoneyPuck)
    model_report.py       metrics table, PvA plots, rolling OOS plot
  draft/
    historical_scoring.py     scores 18 seasons of actuals under pool
       rules
    historical_value_curves.py  aggregates seasons into F/D/G value
       curves
    pool_structure.py         snake-draft pick math, roster caps
    strategy_sim.py           Monte Carlo position-order comparison
    report.py                 writes draft-strategy CSVs/report
    live_state.py             live draft state (duck-types POLICIES)
    player_match.py           fuzzy/typo-tolerant player-name resolution
    live_repl.py              interactive command loop for live_draft.py
  pipeline.py             stage orchestration (scrape/clean/train/
     predict)

run_season.py         CLI entry point (rankings pipeline)
draft_strategy.py     CLI entry point (draft position-order strategy)
live_draft.py         CLI entry point (live draft-day assistant)

<season>/             one directory per draft year, e.g. 2026-27/
  config.yaml           season parameters and scoring rules
  data/{raw,processed}/ scraped + cleaned data (gitignored)
  models/               persisted joblib artifacts (gitignored)
  results/              rankings, XLSX, report, diagnostics CSV (
     gitignored)
  results/draft/        draft-strategy curves, policy comparison, round
                         tables, live draft state (gitignored)
  plots/                PvA and rolling-OOS plots (gitignored)
```

Listing 1: Repository structure

### 2.2 Pipeline stages

The four stages run in sequence via:

```
python3 run_season.py --season 2026-27 --stage all
# or individually:
python3 run_season.py --season 2026-27 --stage scrape
python3 run_season.py --season 2026-27 --stage clean
python3 run_season.py --season 2026-27 --stage train
python3 run_season.py --season 2026-27 --stage predict
```

`scrape` Pull skater/goalie stats, team stats, rosters, and game logs from the NHL API; pull xG/shot-quality/GSAx data from MoneyPuck. Seed from prior season's raw directory to avoid re-downloading unchanged historical seasons.

`clean` Merge position stats with team context; compute per-game rates; output `skater_team_data.csv` and `goalie_team_data.csv`.

`train` Run feature engineering, train the 5-algorithm zoo for each target, select the best model, persist to `models/<target>.joblib`.

`predict` Load persisted models, append prediction rows for the target season, score pool points, write rankings, generate diagnostics. Runs in ∼2s (no retraining).

### 2.3 Environment and running the tools

Every command below is run from the repository root:

```
cd /Users/rcejxd36/Library/CloudStorage/Dropbox/hockeyanalytics
```

**macOS note:** use `python3`, not `python` — on this machine (and modern macOS generally), `python` is not on `PATH` at all (`which python` finds nothing); only `python3` is. Every command in this document uses `python3` for that reason.

**One-time setup** (install dependencies — only needed once, or again after `requirements.txt` changes):

```
python3 -m pip install -r requirements.txt
```

**The three CLI tools**, in the order used each season:

1. `run_season.py` — the main pipeline (scrape, clean, train, predict; see the stages above). Run first each season, before either draft tool.
2. `draft_strategy.py` — position-order strategy (what order to draft *positions*), run once per season before draft day. Full usage in Section 12.
3. `live_draft.py` — the live, player-level assistant, run *during* the actual draft:

   ```
   python3 live_draft.py --season 2026-27 --my-slot 4
   ```

   This is interactive — leave it running in a terminal window for the whole draft. Type `pick <player name>` for every pick as it's announced (yours and every opponent's — this pool's draft is manual/verbal, so there's nothing to pull picks from automatically). Type `help` inside the tool for the full command list (`show`, `top`, `sleepers`, `undo`, `quit`). State saves to disk after every pick, so if the terminal closes or the laptop sleeps, re-running the same command resumes exactly where it left off.

## 3 Data Sources

### 3.1 NHL API (primary)

**Endpoint:** `api.nhle.com` and `api-web.nhle.com`. Free, no authentication key required. Historical data from the 2008-09 season onward.

Data pulled per season: skater counting stats (goals, assists, shots, TOI, hits, blocks, PIM, plus/minus), goalie stats (wins, losses, OTL, shutouts, saves, shots against, GAA, save%), team-level stats (goals for/against, points, games played), and current-season per-game logs.

Roster snapshots are pulled as of `roster_as_of_date` in the season config, so off-season trades, free-agent signings, and waiver moves are reflected automatically.

### 3.2 MoneyPuck (secondary)

**Source:** `moneypuck.com/data.htm` — free CSV downloads, no key required. All 18 seasons (2008–2025) successfully fetched: 81,355 skater rows, 8,510 goalie rows.

MoneyPuck provides metrics unavailable from the NHL API:

- **Expected goals (xG)**: shot-quality-adjusted goal probability per shot
- **Danger-zone shot counts**: high / medium / low danger breakdowns
- **gameScore**: composite single-number performance index
- **GSAx** (goalie-specific): goals saved above expected

**Join strategy:** The integer `player_id` is identical in both sources. Join key: `player_id` (integer) + season start year. No name-matching is used. Match rate: ∼94.5%. The remaining ∼5.5% are fringe players genuinely absent from MoneyPuck's records (very few games played). A name-fallback join is attempted for ID misses only.

**Rate-limit handling:** 1 s between year fetches, 3 s between player entities, 15 s retry on HTTP 429.

### 3.3 Cross-source reconciliation

A dedicated diagnostic script (`common/diagnostics/reconcile.py`) validates agreement between NHL API counting stats and MoneyPuck. Results for the full historical dataset: Pearson $r \geq 0.9999$ for goals, assists, points, and games_played. The few games_played outliers in earlier seasons trace to historical surname collisions in MoneyPuck (e.g. two players named "Jones" in the same season) — the player_id join prevents these from affecting the model.

### 3.4 Sources not used

**Natural Stat Trick**
: Skipped. Free but requires a manually-approved access key before automated pulls. Its stats mostly overlap MoneyPuck's.

**Evolving-Hockey**
: Skipped. The valuable parts (RAPM, GAR, projections, CSV downloads) are paywalled; the free tier adds nothing beyond MoneyPuck.

**Elite Prospects**
: Skipped. Bio/prospect data; not stats-focused; scraping ToS is unclear.

## 4 Feature Engineering

All feature engineering lives in `common/features/engineering.py`.

### 4.1 Lag features

The core predictor of next-season performance is last-season (and two-seasons-ago) performance. All individual statistics, MoneyPuck metrics, and team-context columns are lagged before use:

$$x_{i,t}^{\text{lag}k} = x_{i,t-k}, \quad k \in \{1, 2\} \tag{1}$$

Lags are computed via a vectorized `groupby("player_id").shift(k)`, producing columns `<feature>_lag1` and `<feature>_lag2` for every eligible feature. Rows missing all core lag columns (players with no prior history) are dropped from training.

### 4.2 Leakage prevention

The feature selector uses an **allowlist**, not a denylist. A column is eligible as a model predictor only if it satisfies at least one of:

1. Contains `"lag"` in its name (i.e. it is a genuine lag column), or
2. Contains `"covid"` in its name (COVID season indicators), or
3. Contains any of the `_ENGINEERED_MARKERS` substrings: `_career_avg`, `_hist_avg`, `_hist_std`, `_trend`, `_per_game_lag`, `rel_team`, `teammate`, `normalized_team`, `_performance`, `_x_`

Contemporaneous current-season columns never pass this filter (unless explicitly listed and engineered). This allowlist design catches leakage by construction — a new column added to the data schema cannot slip through unless the author intentionally adds it to the markers list.

### 4.3 Career averages

For high-variance rate statistics, the single most-recent season lag is a noisy estimate. An expanding career average provides a more stable signal:

$$\bar{x}_{i,t}^{\text{career}} = \frac{1}{t-1} \sum_{s<t} x_{i,s} \tag{2}$$

Implemented via `groupby("player_id").shift(1).expanding().mean()` — the shift(1) ensures only seasons prior to $t$ are included (no leakage). Produces columns named `<feature>_career_avg`.

Career averages are computed for:

- **Goalies**: save_pct, GAA, shutouts/game, wins/game, eb_save_pct, GSAx
- **Defense**: plus_minus, plus_minus_per_game

### 4.4 Empirical Bayes save percentage

Raw season save percentage is a binomial proportion subject to substantial sample-size noise. A backup goalie who faced 300 shots and saved 275 (Sv% = 0.917) has a much wider posterior uncertainty than a starter who faced 1,800 shots at the same rate. The Empirical Bayes (EB) approach (Thomas 2006; community standard in hockey analytics) shrinks each estimate toward the league mean in proportion to the goalie's shot volume.

**Model:** Assume saves $\sim$ Binomial$(n, p)$ where $n$ = shots faced and $p$ is the true save rate. Place a Beta prior $p \sim \text{Beta}(\alpha, \beta)$. The posterior mean is:

$$\hat{p}_i^{\text{EB}} = \frac{\alpha + \text{saves}_i}{\alpha + \beta + n_i} \tag{3}$$

**Prior estimation:** $\alpha$ and $\beta$ are estimated from the data via method of moments on all goalie-seasons with $n_i \geq 100$ shots:

$$\hat{\mu} = \text{mean}(\hat{p}_i), \quad \hat{\sigma}^2 = \text{var}(\hat{p}_i) \tag{4}$$

$$\hat{\alpha} = \hat{\mu}\left(\frac{\hat{\mu}(1-\hat{\mu})}{\hat{\sigma}^2} - 1\right), \quad \hat{\beta} = \hat{\alpha}\,\frac{1-\hat{\mu}}{\hat{\mu}} \tag{5}$$

If the pooled dataset contains fewer than 50 qualifying seasons, the fallback prior $(\alpha, \beta) = (85.0,\ 8.5)$ is used, placing the prior mean at $\approx 0.909$ with moderate concentration. The resulting column `eb_save_pct` is then lagged and career-averaged before use.

### 4.5 GSAx and shot-quality features

Raw GAA conflates shot volume (a team-defense quality) with goalie efficiency. MoneyPuck provides expected goals against based on shot location and type, enabling a cleaner individual-skill metric:

$$\text{GSAx}_i = \text{xGoals\_against}_i - \text{goals\_against}_i \tag{6}$$

A positive GSAx means the goalie *outperformed* the shot difficulty they faced. GSAx is more persistent season-to-season than raw save percentage (published YoY autocorrelation: $r \approx 0.15$–$0.30$ for GSAx vs $r \approx 0.05$–$0.15$ for raw Sv%).

Additionally, `hd_shot_pct` = high-danger shots / total shots against captures the shot-mix the defense allowed — a partial control for team quality independent of the goalie.

Both features are lagged and added to career averages.

### 4.6 COVID and experience features

**COVID indicators:** The 2019-20 season was suspended mid-year; the 2020-21 season was shortened to 56 games. A binary `covid_year` flag and interaction terms (`<metric>_x_covid`) are added to allow models to down-weight these atypical seasons.

**Experience:** A polynomial in years played is added for each position:

$$\text{years\_played}, \quad \text{years\_played}^2, \quad \text{years\_played}^3$$

with a position-specific "prime" window (forwards: years 3–10; defense: years 3–12; goalies: years 3–12) encoded as an additional binary indicator.

### 4.7 Interaction features

`engineer_features()` generates pairwise lag×lag interaction terms between a small set of key metrics (e.g. plus_minus_per_game × points_per_game for defense), and lag × COVID interactions for the most volatile stats. These are allowlisted via the `_x_` marker.

## 5 Modeling Framework

### 5.1 Algorithm zoo

Five candidate algorithms are trained for each target. Table 2 summarises the hyperparameter search for each.

Linear models (Ridge, Lasso, ElasticNet) receive a `RobustScaler` prior to fitting. Tree models are scale-invariant and receive no scaling.

Table 2: 5-algorithm zoo and search configuration

| Algorithm | Search | Hyperparameter grid |
|---|---|---|
| Random Forest | Randomized (8) | `n_estimators` $\in \{150, 250\}$, `max_depth` $\in \{12, 18\}$, `min_samples_split` $\in \{15, 25\}$, `min_samples_leaf` $\in \{8, 12\}$, `max_features` $\in \{\sqrt{p}, 0.4\}$ |
| Gradient Boosting | Randomized (10) | `n_estimators` $\in \{150, 250\}$, `max_depth` $\in \{3, 5\}$, `learning_rate` $\in \{0.05, 0.1\}$, `subsample` $\in \{0.8, 0.9\}$, early stopping: `validation_fraction=0.2`, `n_iter_no_change=10` |
| Ridge | Grid | `alpha` $\in \{10, 50, 100, 500\}$ |
| Lasso | Grid | `alpha` $\in \{0.1, 0.5, 1.0, 5.0\}$, `max_iter=3000` |
| ElasticNet | Grid | `alpha` $\in \{0.1, 1.0, 5.0\}$, `l1_ratio` $\in \{0.3, 0.5, 0.7\}$, `max_iter=3000` |

### 5.2 Cross-validation strategy

When the training data has a temporal ordering (season year), a `TimeSeriesSplit` is used, holding out the most-recent block of the training set:

$$n_{\text{folds}} = \min(5,\ \max(3,\ \lfloor n/30 \rfloor)) \tag{7}$$

$$s_{\text{test}} = \min(0.25,\ \max(0.15,\ 40/n)) \tag{8}$$

This ensures a minimum of 40 rows in the held-out test set while never exceeding 25% of the training data. Without temporal ordering, a shuffled `KFold` is used.

### 5.3 Feature selection

When the feature matrix has more than 20 columns, `SelectKBest` with F-statistic univariate regression is applied before fitting:

$$k = \min(50,\ \max(15,\ \lfloor p/3 \rfloor)) \tag{9}$$

where $p$ is the total number of eligible features. This reduces noise features while retaining at least 15 predictors.

### 5.4 Model selection score

After fitting each algorithm, a **selection score** balances test-set performance with overfitting:

$$\text{score} = R^2_{\text{test}} - 0.1 \times \frac{R^2_{\text{train}} - R^2_{\text{test}}}{R^2_{\text{train}}} \tag{10}$$

A model with a high train $R^2$ but poor test $R^2$ is penalised. The algorithm with the highest selection score is chosen as the production model for each target.

### 5.5 Observation weighting and joint-target candidates

Two extensions to the shared harness, both scoped to goalies (see Section 7.3), were added 2026-08-25:

**Observation weighting.** An optional `sample_weight` parameter threads through `_fit_one()`, `train_and_select()`, and `train_all_vs_exclude_latest()` (default `None` preserves prior behaviour for every non-goalie caller). Goalie `build_xy_for()` computes weight = $\min(\text{GP}, 40)/40$, down-weighting 1–5 start backups — who contribute mostly noise — relative to full-season starters. The weight affects only the estimator's `.fit()` call (passed as `sample_weight` into `GridSearchCV/RandomizedSearchCV.fit()`); `SelectKBest` feature selection and all reported test/OOS metrics remain unweighted, so they stay comparable across the change.

**Joint GAA/save% candidate.** GAA and save% are algebraically linked ($r = -0.84$), so a `sklearn.linear_model.MultiTaskElasticNetCV` fit jointly on both targets — enforcing shared feature sparsity — is tried as an additional per-target candidate alongside the single-task zoo (Table 2). It is fit on the full engineered feature set (no `SelectKBest` pre-filter, since the joint L1 penalty already selects features, and pre-filtering by each target's marginal correlation could drop a feature useful only jointly) and, unlike the single-task models, trains *unweighted* — `MultiTaskElasticNet` does not accept `sample_weight`. Each of the two output columns is wrapped in a thin proxy estimator exposing a normal `predict()`/`coef_` interface, so it can be persisted and consumed as an ordinary single-target model; whichever candidate — single-task or joint — wins by Equation 10 is kept, independently per target.

### 5.6 Model persistence

The winning model for each target is saved to `models/<target>.joblib` via `joblib.dump`. The `--stage predict` step loads these artifacts (∼2s) rather than retraining (∼60s). The `TrainedModel` dataclass stores: `target`, `model_type`, `estimator`, `selected_features`, `scaler`, `metrics`, `best_params`, `y_test`, `y_pred_test` (the latter two enabling diagnostic plots without retraining).

## 6 Out-of-Sample Validation

### 6.1 Primary OOS method: train-all vs. exclude-latest

Every target is validated via `train_all_vs_exclude_latest()`:

1. Train **model A** on all available seasons.
2. Train **model B** on all seasons *except* the most recent.
3. Evaluate model B on the held-out most-recent season — this is the out-of-sample (OOS) evaluation.
4. Choose the model with the higher selection score (Eq. 10) for production prediction. In practice, model A (more data) wins most of the time.

### 6.2 OOS metrics

For each model, the following metrics are reported for both the in-sample holdout and the OOS evaluation:

| | |
|---|---|
| $R^2$ | Coefficient of determination. Fraction of variance explained. $R^2 < 0$ means the model is worse than predicting the training-set mean. |
| **MAE** | Mean absolute error. Scale-dependent; reported in same units as the target. |

**Baseline $R^2$** $R^2$ of a constant predictor (training-set mean). Any model with $R^2 < R^2_{\text{baseline}}$ is flagged `beats_baseline=False`.

**Spearman $\rho$** Rank correlation between predicted and actual values. Range $[-1, 1]$; 0 = no ranking information. Directly measures the pool draft objective.

**Concordance** Pairwise ranking accuracy: $C = \Pr[\hat{y}_i > \hat{y}_j \mid y_i > y_j]$. Computed as $(\tau + 1)/2$ where $\tau$ is Kendall's $\tau$. Range $[0, 1]$; $C = 0.5$ = chance.

**Directional accuracy** Fraction of players where the model correctly predicts whether they are above or below the mean: $\text{dir\_acc} = \Pr[\text{sign}(\hat{y} - \bar{\hat{y}}) = \text{sign}(y - \bar{y})]$.

**Bias** Systematic over- or under-prediction: $\bar{\hat{y}} - \bar{y}$.

`top1_match_max` Binary: does the model's argmax (predicted best player) match the actual best player in the OOS year? Analogously `top1_match_min` for the argmin (e.g. the goalie who would win the best-GAA bonus).

**Rationale for rank metrics:** $R^2 \approx 0$ does not mean the model has zero utility for the pool. The draft is a ranking problem — what matters is whether the model correctly orders players relative to each other, not the absolute predicted value. A model with $R^2 = 0.05$ and concordance $= 0.65$ is genuinely useful.

### 6.3 Rolling OOS evaluation

An optional gold-standard time-series OOS is implemented in `rolling_oos_eval()`. For each holdout year $t$ (requiring at least 5 prior training years), the model is trained on all years $< t$ and evaluated on year $t$. This produces an $R^2$ curve over holdout years, visualised in `plots/diagnostics/rolling_oos_all.png`. It is computationally expensive (one full train per holdout year) and is triggered with the `--rolling-eval` flag.

## 7 Position-Specific Models

### 7.1 Forwards

**Target:** `points_per_game` = (goals + assists) / games_played.

**Algorithms:** All five (no restriction).

**Feature set:** All skater individual stats lagged 1 and 2 years; MoneyPuck xG and shot metrics lagged; team context (goals for/against, power-play efficiency) lagged; experience polynomial; COVID interactions. Forwards are split from defensemen before modelling.

**Observations:** All skaters with at least one full year of lag history in the training data (∼9,000–12,000 rows across 2010–2025).

**OOS performance (2026-07-24):** $R^2 = 0.665$, concordance $\approx 0.83$ — the healthiest target in the pipeline. Points are moderately persistent: elite forwards remain elite.

### 7.2 Defense

**Targets:**

- `points_per_game` (goal + assists per game) — same construct as forwards.
- `plus_minus_residual_per_game` = `plus_minus_per_game` − `points_per_game` (2026-08-25, was raw `plus_minus_per_game`) — see "Joint D-men model" below.

**Algorithms:** All five for both targets.

**Leakage:** `allow_contemporaneous_team=False` for both targets. Contemporaneous team stats (e.g. this season's team goals-for) are NaN at prediction time. Lagged team proxies (`team_goals_for_per_game_lag1`, teammate-quality composites) are used instead.

**Additional features:** Career averages for plus_minus and plus_minus_per_game, and four lagged teammate-quality proxies (offensive strength, defensive strength, consistency, elite-team indicator).

**Prediction blending:** Both defense targets are blended post-prediction with 2-year historical per-game averages to correct for model compression at the extremes of the distribution (Section 8). `pool_ranking.py` reconstructs the full plus_minus prediction (residual model's output + the points model's output) before this blend.

**OOS performance:** points/game $R^2 \approx 0.4$, concordance $\approx 0.72$; plus_minus/game (residual target) $R^2 = 0.145$, concordance $\approx 0.64$ (was $R^2 = 0.031$, concordance $\approx 0.54$ for the raw target — see "Joint D-men model" below).

**Joint D-men model (2026-08-25).** A defenseman's own goal or assist is both a personal point and an on-ice goal-for event, so raw `plus_minus_per_game` conflates a player's own offense with the defensive/team-context signal the model is meant to isolate. Modeling the *residual* after removing the points contribution (`plus_minus_residual_per_game` = `plus_minus_per_game` − `points_per_game`) as the target — reconstructing the full prediction at inference time — slots into the existing single-target harness unchanged (lagged/historical points features were already legitimate, non-leaking inputs to the plus_minus model). This is an approximation, not an exact accounting identity: NHL plus/minus excludes power-play goals, which points does not.

### 7.3 Goalies

**Targets:**

| Target | Internal name | Restriction |
|---|---|---|
| Wins per game | `wins_per_game` | All goalies |
| Shutouts per game | `shutouts_per_game` | All goalies |
| Goals against avg | `goals_against_average` | ≥40 GP qualifiers only |
| Save percentage | `save_percentage` | ≥40 GP qualifiers only |

**Algorithm restriction for GAA and save%:** Only Ridge, Lasso, and ElasticNet (`_LINEAR_ONLY`). Season-to-season autocorrelation for both stats is $r \approx 0.09$ (near zero). Tree models fit this noise and produce predictions that cluster around the training mean; regularized linear models correctly regress toward the mean by design.

**Training filter:** All goalies (not just qualified ones) are included in training. Previously, only goalies with $\geq 40$ GP were used, collapsing the training set from ∼1,100 to ∼340 rows. The QUALIFIED_ONLY distinction now applies only at prediction time (bonus eligibility), not at training time.

**Key engineered features for goalies:**

- `eb_save_pct`: Empirical Bayes posterior mean (Section 4.4) — more stable than raw save_pct, especially for low-volume goalies.
- `gsax`: Goals saved above expected (Section 4.5) — lagged GSAx more persistent than raw Sv%.
- `hd_shot_pct`: High-danger shot fraction — partial control for team defense quality.

- Career averages for all rate stats including `eb_save_pct` and `gsax`.

**Observation weighting and joint GAA/save% model (2026-08-25).** See Section 5.5 for the general mechanism. GP-based observation weighting alone meaningfully improved GAA (OOS $R^2$ $-0.203 \rightarrow -0.062$, concordance now $\approx 0.53$) but left save% a degenerate constant predictor (rank metrics undefined). A joint `MultiTaskElasticNetCV` candidate over (GAA, save%) was tried next and won for save% (test $R^2$ $-0.221 \rightarrow -0.046$, now beats the baseline; OOS $R^2$ $-0.406 \rightarrow -0.185$; concordance $\approx 0.52$, was undefined); GAA kept its single-task, weighted model, which already won there.

**OOS performance:** wins/game $R^2 \approx 0.10$; shutouts/game $R^2 \approx 0.05$; GAA $R^2 = -0.062$ (was $-0.203$); save% $R^2 = -0.185$ (was $-0.839$; joint model, see above). The negative $R^2$ for precision stats is expected given near-zero autocorrelation — this is a fundamental data property, not a model failure (Thomas 2006). The bonus only needs correct argmax/argmin among qualified goalies, so concordance $> 0.5$ is the relevant threshold.

# 8 Prediction Blending for Defense

## 8.1 Problem: model compression

Gradient boosting (and to a lesser extent other tree models) tends to compress predictions toward the center of the training distribution. As a concrete example, Cale Makar's points_per_game_lag1 = 1.053 was predicted as 0.762 — the model "averaged him toward the pack." Separately, ElasticNet predicted near-constant plus_minus for almost all defensemen (heavy $\ell_1$ regularization zeroed most coefficients, leaving predictions clustered at $\sim$0).

The result was that elite D-men ranked far lower overall than hockey knowledge would justify: Makar at #44 overall, for example.

## 8.2 Solution: weighted blend with historical average

For each defense target, the model prediction is blended with a 2-year historical per-game average that is already present in the engineered feature set:

$$\hat{y}_i^{\text{blend}} = \alpha \cdot \hat{y}_i^{\text{model}} + (1 - \alpha) \cdot \bar{y}_i^{\text{hist}} \tag{11}$$

If no historical average is available for player $i$ (e.g. a new player in the league), the blend falls back to the raw model prediction.

## 8.3 Calibrating alpha from OOS concordance

The blend weight $\alpha$ is calibrated from out-of-sample concordance. When concordance = 0.5 (chance), the model adds no ranking information and should receive zero weight. When concordance = 1.0 (perfect ranking), the model should receive full weight. The natural calibration is:

$$\alpha = 2 \times (C_{\text{OOS}} - 0.5) \tag{12}$$

Applied values for the 2026-27 season:

| Target | OOS Concordance | $\alpha$ | Interpretation |
|---|---|---|---|
| `defense_points` | $\approx 0.72$ | 0.45 | Model trusted; hist corrects compression |
| `defense_plus_minus` | $\approx 0.57$ | 0.15 | Weak but real signal; hist dominates |

**Effect:** Makar moved from #44 overall → #5 overall after blending.

**Note (2026-08-25):** `defense_plus_minus` concordance here is measured on the *reconstructed* prediction (residual model's output + points model's output, joint D-men model — Section 7.2), not the residual target's own OOS concordance ($\approx 0.64$). The two are different quantities: prediction errors from the independently-fit points and residual models compound when summed, so the reconstructed quantity's own fit against actual `plus_minus_per_game` is weaker than the residual target's fit against the residual. $\alpha$ is calibrated on the former, since that is what is actually being blended.

# 9 Pool Scoring and Ranking

## 9.1 Projected games

Each player's projected games played is derived from a 3-year rolling average of their actual `games_played_player`, capped at `cfg.games_per_season`.

**Partial-season filter:** Historical seasons with GP $< 25$ are excluded from the rolling window before computing the average. This prevents late-season rookie call-ups (e.g. a player who plays 15 games in their debut year after being recalled from the AHL in March) from biasing the projection downward. The most-recent season is *always* included regardless of GP — a genuine recent injury should still reduce the projection.

$$\mathcal{H}_i = \{\mathrm{GP}_{i,s} : s < t_{\max},\ \mathrm{GP}_{i,s} \geq 25\} \quad \text{(filtered historical seasons)} \tag{13}$$

$$\mathrm{GP}_i^{\mathrm{proj}} = \min\left(\frac{1}{|\mathcal{W}_i|}\sum_{s\in\mathcal{W}_i}\mathrm{GP}_{i,s},\ \texttt{games\_per\_season}\right), \quad \mathcal{W}_i = (\mathcal{H}_i \cup \{\mathrm{GP}_{i,t_{\max}}\})\ \text{tail-3} \tag{14}$$

For players with no qualifying history: forwards and defense default to 70 games; goalies default to 20 games.

**GP-projection diagnostic:** On every predict run, a diagnostic compares each player's projected GP to their most-recent actual GP. Players where $\mathrm{GP}_i^{\mathrm{proj}} < 0.70 \times \mathrm{GP}_i^{\mathrm{last}}$ are logged as anomalies and written to `results/diagnostics/gp_projection_check.csv` (all players sorted by ratio, with columns: player_id, player_name, position_group, projected_games, last_season_gp, ratio). After the partial-season fix, only 13 players remain flagged — all with genuinely irregular career patterns (multi-season injuries, suspensions, retirements) rather than call-up artifacts.

## 9.2 Dual scoring outputs

Two pool-point totals are produced for every player:

- `pool_points`: uses per-player projected GP (injury-risk-adjusted)
- `pool_points_full_season`: uses flat `games_per_season` $= 84$ for all players

Rankings in the output files are sorted by `pool_points` by default.

## 9.3 Forward scoring

$$\mathrm{pool\_points}_i^{\mathrm{fwd}} = \hat{y}_i^{\mathrm{pts/g}} \times \mathrm{GP}_i^{\mathrm{proj}} \tag{15}$$

### 9.4 Defense scoring

$$\widehat{\text{pts}}_i = \hat{y}_i^{\text{pts/g}} \times \text{GP}_i^{\text{proj}} \tag{16}$$

$$\widehat{\text{pm}}_i = \hat{y}_i^{\text{pm/g}} \times \text{GP}_i^{\text{proj}} \tag{17}$$

$$\text{pool\_points}_i^{\text{def}} = \widehat{\text{pts}}_i + \widehat{\text{pm}}_i \tag{18}$$

Blended predictions are used for $\hat{y}^{\text{pts/g}}$ and $\hat{y}^{\text{pm/g}}$.

### 9.5 Goalie scoring

Let $W_i = \hat{y}_i^{\text{wins/g}} \times \text{GP}_i^{\text{proj}}$ and $S_i = \min(\hat{y}_i^{\text{so/g}} \times \text{GP}_i^{\text{proj}},\ W_i)$ (shutouts capped at wins):

$$\text{pool\_points}_i^{\text{goalie}} = 1{\cdot}W_i + 3{\cdot}S_i + 10{\cdot}\mathbf{1}[\text{best GAA among qualified}] + 10{\cdot}\mathbf{1}[\text{best Sv\% among qualified}] \tag{19}$$

The shutout bonus is additive on top of the win point: a shutout win is worth 4 total (1 for the win + 3 shutout bonus), not 3.

A goalie is *qualified* for the bonus if $\text{GP}_i^{\text{proj}} \geq 40$. The GAA bonus goes to $\arg\min_i \hat{y}_i^{\text{GAA}}$ among qualified goalies; the save% bonus to $\arg\max_i \hat{y}_i^{\text{sv\%}}$.

### 9.6 Breakout score (dark-horse detection)

Added 2026-08-25: a `breakout_score` and `dark_horse` flag on every player, answering "who's an overlooked late-round upside pick" — computed entirely from engineered columns that already exist (no new scraping, no training-pipeline change), in `pool_ranking._add_breakout_score()`. Two signals, each already leakage-safe and multi-year-averaged:

1. **Model-vs-history divergence**: $\hat{y}_i^{\text{pred}} - \overline{y}_i^{\text{hist}}$ — the same `*_hist_avg` reference column already used by the defense blend (Section 8), exposed as a standalone signal: how much more the model predicts than the player's own recent track record.
2. **Underlying-metric regression candidate**: for forwards/defense, $\overline{\text{xG}}_i^{\text{hist}} - \overline{\text{goals}}_i^{\text{hist}}$ using MoneyPuck's individual expected-goals column (already joined and auto-lagged for skaters, Section 3) — positive means a player has recently scored fewer goals than their shot quality implies, a "due for positive regression" signal. For goalies, `gsax_hist_avg` directly (already skill-adjusted; no subtraction needed).

Both signals are z-scored within their position group (different units/scale, and across positions) and averaged: $\text{breakout\_score}_i = \frac{1}{2}(z(\text{divergence}_i) + z(\text{luck}_i))$. `dark_horse` flags $\text{breakout\_score}_i >$ 1.0 *and* a predicted rank outside an already-obviously-good cutoff (rank $>$ 30 forwards, $>$ 15 defense, $>$ 8 goalies) — a top-5 player trending up is not a hidden gem, so the flag is gated by rank in addition to the raw score, which is still written to output either way.

**Caveats**: (1) no ADP/consensus draft-position data exists anywhere in this pipeline, so "late round" is necessarily proxied via the model's own predicted rank, not a real "will still be there in your round" guarantee. (2) The regression-candidate signal can't distinguish genuine repeatable elite shooting talent from one-off shooting variance — an elite sniper who reliably outscores their xG will show a *negative* luck z-score by this formula (and did, in the first real run: MacKinnon, Kucherov, Draisaitl all score strongly negative overall) — this is an inherent simplification, not a bug, and matches why the very top of the rankings never gets flagged `dark_horse` regardless of the rank gate.

**Live-draft integration**: since `live_draft.py`'s available-player pool is just this rankings data with picked players removed, both columns flow through automatically — a fire-emoji marker annotates `dark_horse` players inline in every "top available" listing (the recommendation panel and `top`), and a dedicated `sleepers [position] [n]` command shows only dark-horse candidates, sorted by `breakout_score`, available at any point in the draft (no round-based gating).

# 10 Diagnostics

Model diagnostics are generated automatically at the end of every `--stage predict` run.

## 10.1 Outputs

`results/diagnostics/model_metrics.csv`
: One row per model. Columns: model, label, algorithm, n_train, n_test, test_r2, train_r2, baseline_r2, beats_baseline, overfitting_ratio, mae, rmse, cv_mean, spearman_r, concordance, directional_acc, bias, oos_r2, oos_mae, oos_n, oos_chosen_model, oos_spearman_r, oos_concordance, oos_directional_acc, oos_bias, oos_top1_match_max, oos_top1_match_min.

`plots/diagnostics/pva_<target>.png`
: Two-panel plot per model: (left) predicted vs. actual scatter with $y = x$ reference line and $R^2$ annotation; (right) residuals vs. predicted (checks heteroskedasticity). Generated from `y_test`/`y_pred_test` stored in the joblib artifact — no retraining required.

`plots/diagnostics/rolling_oos_all.png`
: Grid of OOS $R^2$ by holdout year for each model (one subplot per model). Shows whether model quality has been stable over time or has degraded in recent seasons.

`results/diagnostics/gp_projection_check.csv`
: All players sorted by $\text{GP}^{\text{proj}}/\text{GP}^{\text{last}}$. Players below 0.70 are flagged in the log as anomalies requiring manual review before draft day. Produced by `pool_ranking._write_gp_diagnostic()`.

`results/diagnostics/source_reconciliation.csv`
: NHL API vs. MoneyPuck agreement on counting stats. Produced by `common/diagnostics/reconcile.py`.

`plots/feature_importance_<target>.png`
: Horizontal bar chart of the top 15 features by importance (tree models: built-in feature importances; linear models: |coef|).

## 10.2 Console summary

A compact table is printed to stdout at the end of every predict run:

```
=== Model fit summary ===
label         algorithm      test_r2  oos_r2  concordance
   oos_concordance
Fwd Points/G  grad_boosting  0.712    0.665   0.912        0.897
Def Points/G  ridge          0.531    0.402   0.869        0.851
...
```

# 11 Results

## 11.1 Model performance

Table 3 summarises out-of-sample performance for all seven production models as of the 2026-08-25 model-fit pass (observation weighting, the joint GAA/save% candidate, and the joint D-men residual model — Sections 5.5, 7.2, 7.3), exact values from `model_metrics.csv`.

Table 3: OOS model performance — all 7 targets (2026-08-25)

| Target | Algorithm | OOS $R^2$ | OOS Conc. | OOS Spear. $\rho$ | Notes |
|---|---|---|---|---|---|
| Fwd Points/G | Gradient Boosting | 0.665 | 0.775 | 0.724 | Healthy |
| Def Points/G | Gradient Boosting | 0.396 | 0.723 | 0.596 | Post-blend |
| Def +/−/G | Ridge (resid.) | 0.145 | 0.642 | 0.402 | Joint D-men model |
| Goalie Wins/G | Ridge | 0.015 | 0.548 | 0.156 | Team-driven |
| Goalie SO/G | Lasso | −0.139 | — | — | Rare event; constant predictor |
| Goalie GAA | Lasso | −0.062 | 0.533 | 0.104 | Weighted; below mean, expected |
| Goalie Sv% | MultiTask ElasticNet | −0.185 | 0.508 | 0.027 | Joint w/ GAA |

## 11.2 Top-10 overall ranking (2026-08-25)

| Overall Rank | Player | Position |
|---|---|---|
| 1 | N. MacKinnon | F |
| 2 | N. Kucherov | F |
| 3 | C. McDavid | F |
| 4 | L. Draisaitl | F |
| 5 | C. Makar | D |
| 6 | D. Pastrnak | F |
| 7 | E. Bouchard | D |
| 8 | N. Suzuki | F |
| 9 | M. Necas | F |
| 10 | M. Celebrini | F |

Cale Makar at #5 reflects the defense blending fix; prior to blending he ranked #44.

# 12 Draft Position-Order Strategy

The player-ranking pipeline above answers *which players* are most valuable. It says nothing about *in what order to draft positions* during the live 9-team snake draft, given that the best player at a chosen position is assumed to be taken and that other teams' picks are unpredictable. This section documents `draft_strategy.py` and the `common/draft/` package, added 2026-08-24, which answers that question directly. It is a positional (not player-level) strategy tool: it never tracks or recommends specific players, and is explicitly scoped separately from the still-unbuilt live, player-tracking draft-day assistant (Future Work).

Table 4: 9-team pool roster structure

| | Starters | Bench/IR | Total per team |
|---|---|---|---|
| Forward | 7 | 3 | 10 |
| Defense | 3 | 2 | 5 |
| Goalie | 1 | 1 | 2 |
| League-wide (9 teams): 90 forwards, 45 defensemen, 18 goalies drafted, 17 rounds. | | | |

### 12.1 Pool structure

Bench/IR slots are substitutable into the active lineup up to twice a week (unlimited for injuries) — generous enough that every drafted player is treated as contributing full value, so "cap" above is starters + bench combined rather than starters alone. The draft order is a single random ordering fixed before the draft and reversed each round (standard snake).

### 12.2 Historical value curves

Rather than relying on a single season (either an actual-results season or the current projections alone), a value curve per position is built from **18 real NHL seasons, 2008-09 through 2025-26**, scored under the pool's own rules via `compute_pool_points()` (Section 9) reused unchanged — only the input columns are adapted, mapping each historical season's actual stats onto the `pred_*/projected_games` names that function expects.

**Season-length normalization.** Three seasons were not 82 games: 2012-13 (lockout, 48 games), 2019-20 (COVID suspension, ≈70 games league-wide average), and 2020-21 (COVID, 56 games). Counting stats are rate-converted and rescaled to an 82-game-equivalent pace:

$$\mathrm{GP}^{\mathrm{norm}}_{i,s} = 82 \times \frac{\mathrm{GP}_{i,s}}{L_s}, \qquad L_s = \begin{cases} 48 & s = \text{2012-13} \\ 70 & s = \text{2019-20} \\ 56 & s = \text{2020-21} \\ 82 & \text{otherwise} \end{cases} \tag{20}$$

Using $\mathrm{GP}^{\mathrm{norm}}$ as *both* the pace-scaling factor and the goalie bonus-qualification threshold (rather than raw $\mathrm{GP}_{i,s}$) has a second effect: a goalie who played all 48 games of the 2012-13 lockout season is correctly treated as having played a "full" season, not penalized against the fixed 40-game bar meant for an 82-game season.

For each season, players are ranked within position by pool_points; the final curve is the equal-weighted average pool_points at each rank across all seasons with a player at that rank — the same "trust the multi-year pattern over any one year" principle already used elsewhere in the pipeline (2-3 year lag averaging in `common/features/engineering.py`, the 3-year GP window in `pool_ranking._projected_games()`), applied here across the full 18-year history to build a stable scarcity curve rather than a short-term trend.

**Plausibility cross-check:** the historical curve's shape (each rank's value normalized to a fraction of the #1 player's value) is correlated against the current 2026-27 projected ranking's shape. A plain rank-vs-rank (Spearman) correlation would be trivially 1.0 for any two monotonically-decreasing curves regardless of whether their actual shapes agree, so a Pearson correlation on the normalized values is used instead:

Table 5: Historical value-curve vs. current-season projection shape agreement

| Position | Normalized-shape Pearson $r$ |
|---|---|
| Forward | 1.00 |
| Defense | 1.00 |
| Goalie | 0.93 |

Table 6: Positional scarcity, 18-season aggregated curves (2026-08-25)

| Position | #1 value | Range drop (#1→last drafted) | Cliff (15 spots beyond) | Verdict |
|---|---|---|---|---|
| Forward | 118.6 | 56% (to #90: 51.7) | 7% (#105: 48.2) | Safe to defer |
| Defense | 100.9 | 61% (to #45: 39.7) | 17% (#60: 33.2) | Steep, but a fallback exists |
| Goalie | 72.7 | 51% (to #18: 35.3) | **36%** (#33: 22.4) | **Don't wait** |

### 12.3 Scarcity: two distinct signals

The *range drop* (#1 to the last player the league will draft at that position) measures overall depth; the *cliff* (last drafted vs. 15 spots further down) measures the real draft-day risk of waiting one round too long. Defense has the steepest range drop, but forwards and defense both still have a reasonable fallback tier just past their cutoffs. Goalies are the clear outlier: only a 51% range drop, but a further 36% collapse in the 15 picks past #18 — missing the goalie window is expensive, which matches standard hockey-pool intuition (shallow starter-caliber goalie pool) even though the raw range-drop numbers alone would not have shown it.

### 12.4 Draft simulation and strategy comparison

Opponents (8 other teams) are modeled per the simplification confirmed for this tool: at each pick, a team chooses among its still-open positions via a softmax over each eligible position's current top-of-curve value (temperature $\tau = 1.5$ by default), approximating "best player available among remaining needs" without deeper per-team modeling. The user's own picks are generated by one of three candidate policies, compared via $M = 1500$ Monte Carlo drafts per policy per draft slot.

**Starters-before-bench draft mechanic (fixed 2026-08-25):** every team — opponents included, not just the user — must fill all 11 starter slots (7F/3D/1G, in any order/mix among still-open starter categories) before any of the 6 bench slots (3F/2D/1G) becomes draftable; e.g. a 2nd goalie is never "still-open" until the starter F/D/G slots are all full, even though total goalie capacity would otherwise allow it. A prior version of this simulation omitted this constraint (only enforcing the *combined* starters+bench cap per position), which let recommendations illegally draft a 2nd goalie as early as round 11 — see `PROJECT.md`'s changelog for the concrete before/after trace. `eligible()` in `strategy_sim._State` now restricts each team to starter-open positions until all 11 starter slots are filled, then to any still-under-cap position; every result below reflects the corrected constraint.

`value_greedy` Always take the eligible position with the highest current top-of-curve value (naive best-player-available, position-blind beyond eligibility).

`balanced_need` Take the eligible position furthest below its final roster target fraction (10F:5D:2G) — even pacing across the draft.

`urgency_greedy` VONA-style: take the position with the largest estimated value erosion by the

user's own next pick, where the expected number of that position taken by opponents in the interim is estimated from its share of remaining league-wide need.

Table 7: Best-performing policy by draft slot (avg. simulated team pool_points total)

| Slot | value_greedy | balanced_need | urgency_greedy | Winner |
|---|---|---|---|---|
| 1 | 1079.5 | 1077.5 | **1084.3** | urgency_greedy |
| 2 | 1067.2 | 1068.0 | **1070.3** | urgency_greedy |
| 3 | 1061.5 | 1064.3 | **1068.6** | urgency_greedy |
| 4 | 1060.4 | 1062.1 | **1068.3** | urgency_greedy |
| 5 | 1057.9 | **1062.3** | 1054.2 | balanced_need |
| 6 | 1054.6 | **1061.9** | 1052.1 | balanced_need |
| 7 | 1050.5 | **1060.9** | 1057.8 | balanced_need |
| 8 | 1050.8 | **1060.7** | 1055.3 | balanced_need |
| 9 | 1051.8 | 1062.8 | **1063.4** | urgency_greedy |

**Result: the winning policy now splits by draft position.** `urgency_greedy` wins slots 1–4 and slot 9 (five of nine), decisively at slots 1–4 (2.3–6.8 points ahead of the runner-up) but only barely at slot 9 (1063.4 vs. 1062.8, a 0.6-point practical tie). `balanced_need` wins slots 5–8 (four of nine), also decisively (3.1–7.3 points ahead). `value_greedy` never wins any slot. This is a genuinely different, more nuanced picture than a single universal recommendation: early slots (1–4) wait the longest for their next pick under snake-draft reversal, where `urgency_greedy`'s explicit modeling of opponent scarcity erosion in that gap pays off; mid-table slots (5–8) have more evenly-spaced picks, where `balanced_need`'s simpler even-pacing heuristic is enough. Slot 9 (the last pick of each odd round, immediately followed by the first pick of the next even round) is a genuine toss-up between the two. **Recommendation: use `urgency_greedy` at slots 1–4, `balanced_need` at slots 5–8, and either at slot 9** — `live_draft.py` already looks up the per-slot winner from this table automatically ('policy_comparison.csv').

### 12.5 Full recommended draft order — all 9 slots

Rather than one representative slot, Table 8 lays out the full round-by-round recommended position for every draft slot side by side — a complete reference, and also the clearest way to see the pattern across the whole pool at a glance.

**Lessons learned from the full grid** (updated 2026-08-25 after fixing the starters-before-bench draft mechanic described above — the previous version of this table let the simulation illegally draft a 2nd goalie as early as round 11, which materially changed both the policy comparison and the shape of the recommendations below):

- **Every slot now hits exactly the required 7F/3D/1G starter mix by round 11, no exceptions** — this is the fix itself made visible: columns 1–9, rows 1–11 always sum to 7 forwards, 3 defense, 1 goalie, whichever policy wins that slot. The old, buggy grid showed a 2nd goalie inside the first 11 picks at several slots; that can no longer happen.
- **Which policy wins now genuinely depends on where you pick, not a single universal answer.** `urgency_greedy` wins slots 1–4 and 9 (Table 7) — decisively at 1–4, since those slots wait the longest for their next pick under snake-draft reversal, where explicitly modeling opponent scarcity erosion pays off. `balanced_need` wins slots 5–8, where picks are more evenly spaced and simple pacing is enough. The four `balanced_need` slots (5, 6, 7, 8) still share one identical sequence, `F D G F F D F F D F F G D F F D F` at 100% confidence — a single memorable recipe, but now only for those four slots, not seven. Slots 3 and 4 (both

`urgency_greedy`) also converge on nearly the same sequence as each other; slots 1, 2, and 9 each diverge more, and slot 1 in particular is noticeably less certain (confidence dips as low as 54%), since `urgency_greedy` is estimating opponent behavior rather than reacting only to the user's own roster.

- **The 2nd goalie's timing is the clearest behavioral difference between the two policies.** Under `balanced_need` (slots 5–8) it's taken at round 12 — the earliest legal bench pick, immediately once the bench phase opens. Under `urgency_greedy` it's pushed much later — round 14 at slots 2–4, round 15 at slot 1, round 16 at slot 9 — because by then the 1st-goalie scarcity pressure that drove the early pick (secured by round 2–3 everywhere, well ahead of the #18 cutoff and its 36% post-cutoff cliff, Table 6) has already been resolved, and a bench goalie carries little further urgency. Forwards remain the deepest, most cushioned position and dominate the back half of the draft regardless of policy.

Table 8: Full recommended draft order by round, all 9 slots (F=forward, D=defense, G=goalie)

| Round | Slot | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 1 | F | D | D* | D* | F | F | F | F | F |
| 2 | F | D | G | G* | D | D | D | D | G* |
| 3 | G* | G* | D | D | G | G | G | G | F |
| 4 | F | D* | D* | D* | F | F | F | F | D* |
| 5 | D* | F* | F* | F* | F | F | F | F | F |
| 6 | F | F | F | F | D | D | D | D | D* |
| 7 | D | F | F | F | F | F | F | F | F |
| 8 | F | F | F | F | F | F | F | F | D* |
| 9 | D | F | F | F | D | D | D | D | F |
| 10 | F | F | F | F | F | F | F | F | F |
| 11 | F | F | F | F | F | F | F | F | F |
| 12 | F | D | D* | D | G | G | G | G | D |
| 13 | D | D | D* | D | D | D | D | D | F |
| 14 | F | G | G* | G | F | F | F | F | D* |
| 15 | G* | F | F | F | F | F | F | F | F |
| 16 | F | F | F | F | D | D | D | D | G* |
| 17 | D* | F | F | F | F | F | F | F | F |

Forward Defense Goalie *below 100% simulated confidence (slots 5–8 are 100% throughout; slots 1–4 and 9 vary, see text)

Exact overall pick numbers per slot (which differ due to snake-draft math even where the position sequence is identical) remain in `results/draft/round_priority_slot_<1-9>.csv`, one file per slot, as generated by `draft_strategy.py`.

### 12.6 Usage and outputs

```
python3 draft_strategy.py --season 2026-27              # all 9 slots
python3 draft_strategy.py --season 2026-27 --my-slot 5   # + slot-5
    table
```

Writes to `results/draft/`: `value_curves.csv` (full F/D/G curves), `policy_comparison.csv` (all slots × policies), `round_priority_slot_<1-9>.csv` (one per slot), and `draft_strategy_report.txt` (the summary printed above). No changes were made to `run_season.py`, `pool_ranking.py`, or the scrape/clean/ train/predict pipeline — this tool only reads the raw historical CSVs and the current rankings file already produced by that pipeline.

# 13 Known Limitations

**Goalie precision stats**

Season-to-season autocorrelation for GAA and save% is $r \approx 0.09$ — this is a fundamental property of goalie performance data, not a model failure. Published ranges (Thomas 2006; Macdonald 2010; Schuckers & Curro 2013) are $r \approx 0.05$–$0.15$ for raw Sv% and $r \approx 0.15$–$0.30$ for GSAx. No model can reliably predict who will have the best GAA next season from prior stats alone. The pool bonus only requires correctly identifying the argmax/argmin among a small number of qualified goalies, which is a weaker requirement than accurate regression.

**Defense plus/minus**

Modeled as a residual after removing the points contribution (2026-08-25): the residual target's own OOS $R^2$ improved from 0.031 to 0.145 and concordance from $\approx 0.54$ to $\approx 0.64$. Still a comparatively weak signal — this stat is highly dependent on team context (a plus/minus changes by $\pm 1$ whether or not the goalie saves a shot that would not be attributed to any on-ice skater) — and the residual formulation is an approximation, not an exact accounting identity (NHL plus/minus excludes power-play goals, points doesn't). Prediction blending (alpha recalibrated to 0.15, Section 8) applies to the *reconstructed* (residual + points) prediction, whose own OOS concordance ($\approx 0.57$) is lower than the residual target's ($\approx 0.64$) — the two independently-fit models' errors compound when summed.

**Traded players — historical team context**

Players who changed teams during a historical season are assigned the *first* team listed in the NHL API's comma-joined `team_abbrev` field. Season totals (goals, assists, games_played) are correctly preserved. Only the team-context lag features (used by the defense model) may be inaccurate for multi-team seasons. Per-team game logs are only available for the most recent season, so a majority-team fix across all 18 training seasons would require expanding game-log history.

**MoneyPuck coverage**

$\sim$5.5% of players are unmatched in MoneyPuck. These are fringe players with very few games played who are genuinely absent from MoneyPuck's records, not join failures. They receive NaN for all MoneyPuck features, which become 0 after imputation.

**Pool scoring rules constant year-to-year**

No time-varying config is needed; the rules have not changed. Any future rule change requires only a config edit.

**Draft strategy: unvalidated modeling choices**

Section 12's season-length normalization (82-game-pace rescaling) and opponent policy (softmax over eligible-position top value, $\tau = 1.5$) are reasonable but new assumptions with no prior art in this codebase and no historical draft-pick data to calibrate against. The `urgency_greedy` policy's expected-picks-at-position estimate is a share-of-remaining-need approximation, not an exact computation. Logging this season's actual draft (once the deferred live tool exists) would enable empirical calibration in future seasons.

# 14 Future Work

**Done, 2026-08-25: observation weighting, joint GAA/save% model, joint D-men model**

Three items previously listed here are now implemented — see the Modeling Framework (Section 5) and Position-Specific Models (Section 7) sections, and `PROJECT.md`'s 2026-08-25 changelog entry for full detail and before/after metrics: goalie observation weighting (`sample_weight = min(GP, 40) / 40`, threaded through the shared training harness), a `MultiTaskElasticNetCV` joint (GAA, save%) candidate (kept only for save%, where it beat the single-task model), and the joint D-men model (plus_minus modeled as a residual after removing the points contribution).

**GP diagnostic Layer 2: game-log debut date**

The current partial-season filter (GP $< 25$) works on the aggregate season count. A richer signal is already available: `nhl_game_logs.csv` contains `gameDate` in ISO format; the existing `get_player_game_log()` endpoint in `nhl_api.py` can fetch any player/season combination. For any player whose debut season has GP $< 50$, fetching their game log and checking the first game date would definitively distinguish a March call-up (first game after February) from an October injury (first game in November). Deferred because the threshold filter handles the known cases; implement if edge cases emerge.

**Majority-team for historical traded players**

Expand per-game log collection from the current season only to all historical seasons (one API call per player per season for $\sim$18 years). Assign each player to the team they played the most games for. This would make the team-context lag features more accurate for the $\sim$5–10% of historical rows involving mid-season trades.

**Done, 2026-08-25: interactive draft-day tool**

`live_draft.py` (backed by `common/draft/live_state.py`, `player_match.py`, `live_repl.py`) is a live, player-level CLI assistant: picks (the user's and every opponent's — the league's draft is manual/verbal, no platform to pull from) are entered by hand with typo/partial-name-tolerant matching, and on the user's turn it recommends a player using the same position-order policy logic as Section 12 — reused unchanged against live availability instead of the historical Monte Carlo simulation. See `PROJECT.md`'s 2026-08-25 changelog entry for full detail; a fuller write-up here is deferred until after first real-draft use.

**2027-28 season**

Copy `2026-27/config.yaml` → `2027-28/config.yaml`, update `season`, `season_id`, `roster_as_of_date`, `games_per_season` (if the NHL expands again), and any `trade_overrides`. Run `python3 run_season.py --season 2027-28 --stage`

`all`.

## References


- Thomas, A. C. (2006). "The impact of puck possession and location on ice hockey strategy." *Proceedings of the Joint Statistical Meetings (Statistics in Sports)*. [Empirical Bayes save% shrinkage; near-zero YoY autocorrelation for Sv%.]
- Macdonald, B. (2010). "A regression-based adjusted plus-minus statistic for NHL players." *Journal of Quantitative Analysis in Sports*, 7(3).
- Schuckers, M. & Curro, J. (2013). "Total Hockey Rating (THoR): A comprehensive statistical rating of National Hockey League forwards and defensemen based upon all on-ice events." *MIT Sloan Sports Analytics Conference*.
- Gramacy, R. B., Jensen, S. T., & Taddy, M. (2013). "Estimating player contribution in hockey with regularized logistic regression." *Journal of Quantitative Analysis in Sports*, 9(1).
- MoneyPuck.com methodology notes: `https://moneypuck.com/about.htm`. [GSAx definition; shot danger zone classification.]